

SOFTWARE CONSTRUCTION AND DEVELOPMENT

Engr. Muhammad Umer Haroon

SOFTWARE CONSTRUCTION AND DEVELOPMENT

WEEK 6

RECALL LAST WEEKS

Regular Expressions

- Regular expressions are one of the most powerful and most misunderstood tools in a software developer's toolkit. They appear everywhere in software construction, from validating user input to parsing log files to searching and replacing text in source code.

REGEX

- A regular expression, often shortened to regex, is a pattern that describes a set of strings. It is a mini language for matching text.
- Instead of writing long chains of if statements and string methods to check whether a piece of text looks like an email address or a phone number, you describe the shape of the text using a pattern, and a regex engine checks whether a given string matches that shape.

Regex can be used in most places

- Regular expressions are not unique to Python. They exist in almost every programming language, in command line tools and in text editors such as VS Code. The syntax is mostly shared, with small differences between languages

Regular Expressions Matter in Software Construction

- In software construction and development, text is everywhere. Configuration files are text. Log files are text. User input is text. Source code is text. Data formats such as JSON, CSV, and XML are text. Being able to describe, search, and manipulate text patterns efficiently is a core skill.

Some common uses in real projects

include validating user input such as email addresses, phone numbers, and passwords; extracting fields from log files such as timestamps, IP addresses, and error codes; searching and replacing patterns across a codebase; parsing simple configuration files and data formats; sanitizing input before storing it in a database; and generating tokens, slugs, and identifiers from human readable strings.

Pattern Matching:

- Regular expressions provide a powerful way to search for specific patterns within strings.
- This is essential in many software applications where data needs to be validated or extracted from user input or files.
- For example, regex can be used to validate email formats, phone numbers, or other structured data inputs

Text Processing

- Regex is widely used in text processing tasks such as searching, replacing, and splitting strings.
- This functionality is crucial for applications that manipulate text data, such as text editors, command-line tools

Input Validation

- In software development, ensuring that user input meets certain criteria is vital for maintaining data integrity and security.
- Regular expressions can enforce rules on what constitutes valid input, such as ensuring passwords meet complexity requirements or that usernames do not contain special characters

Lexical Analysis

- Regular expressions are fundamental in the field of compilers and interpreters for programming languages.
- They are used in lexical analysis to tokenize input strings into meaningful components (tokens), which are then parsed according to the grammar of the language

The Python re Module

- Python provides regular expression support through the standard library module called re.

Functions in re module

- re.search() Searches for a pattern in a string and returns a match object.
- re.findall() Returns all occurrences of a pattern in a string as a list.
- re.sub() Replaces occurrences of a pattern with a specified string.
- re.split() Splits a string by the occurrences of a pattern.

Meta characters

. ^ $ * + ? { } [ ] \ | ( )

Meta Characters

- Matches any character except newline.

^ Matches the start of a string.

$ Matches the end of a string.

***** Matches 0 or more repetitions of the preceding element.

+ Matches 1 or more repetitions.

? Matches 0 or 1 repetition.

{m,n} Matches between m and n repetitions.

Character classes are things like [0-9], [a-z], [A-Z], which explicitly list ranges or sets of characters.

Metacharacters are shorthand symbols that represent common character classes without writing them out.

Character Classes

\d: Matches any digit (equivalent to [0-9]).

\D: Matches any non-digit character.

\w: Matches any alphanumeric character (equivalent to [a-zA-Z0-9_]).

\W: Matches any non-alphanumeric character.

\s: Matches any whitespace character (spaces, tabs).

\S: Matches any non-whitespace character.

- Metacharacters (except \) are not active inside classes.
- [ ] represent character class

Groups and Ranges

- Use parentheses () to create groups and square brackets [] for ranges

CODE

```
# Example 1: Find all lowercase letters (a-z) in the input string
output = re.findall("[a-z]", input_string)
print("Output of [a-z]:", output)
```

```
# Example 2: Find all uppercase letters (A-Z) in the input string
output = re.findall("[A-Z]", input_string)
print("Output of [A-Z]:", output)
```

```
# Example 3: Find all occurrences of an uppercase letter followed by a lowercase letter
output = re.findall("[A-Z][a-z]", input_string)
print("Output of [A-Z][a-z]:", output)
```

```
# Example 4: Find all occurrences of an uppercase letter followed by one or more lowercase letters
output = re.findall("[A-Z][a-z]+", input_string)
print("Output of [A-Z][a-z]+:", output)
```

```
# Example 5: Find all occurrences of an uppercase letter followed by zero or more lowercase letters
output = re.findall("[A-Z][a-z]*", input_string)  # The '*' means zero or more occurrences of the lowercase letters
print("Output of [A-Z][a-z]*:", output)
```

```python
# Example 6: Find all characters that are NOT uppercase letters (negation)
output = re.findall("[^A-Z]", input_string)   # The '^' inside brackets negates the character class
print("Output of [^A-Z]:", output)
```

```
# Example 7: Find sequences of characters that do not contain uppercase letters (one or more)
output = re.findall("[^A-Z]+", input_string)  # Matches one or more characters that are not uppercase letters
print("Output of [^A-Z]+:", output)
```

```
# Search for the first occurrence of 'dog' in the text
match = re.search(pattern, text)
if match:
    print("Found:", match.group())  # Print the matched string if found
```

```
    # Find all occurrences of 'dog' in the text
matches = re.findall(pattern, text)
print("All matches:", matches)  # Print all matches found
```

```python
# Substitute all occurrences of 'dog' with 'Puppy'
new_text = re.sub(r"dog", "Puppy", text)
print("Modified text:", new_text)  # Print the modified text
```

```python
# Split the text into parts based on whitespace characters
parts = re.split(r"\s+", text)
print("Split parts:", parts)  # Print the list of split parts
```

```python
# UNDERSTANDING USE OF ?

# Sample input string
input_string2 = "The cat sat on the mat. The cats are cute."

# Use regex to find occurrences of 'cat' optionally followed by 's'
pattern = r"cats?"  # Matches 'cat' or 'cats'

# Find all matches in the input string
matches = re.findall(pattern, input_string2)

# Print the matches found
print("Matches found:", matches)
```

```python
# UNDERSTANDING OF {}

# Sample input string
input_string = "aaa aa aaaaaa aa aaaaa aaa"

# Use regex to find sequences of 'a' that occur between 2 and 4 times
pattern = r"a{2,4}"  # Matches 'a' appearing 2 to 4 times

# Find all matches in the input string
matches = re.findall(pattern, input_string)

# Print the matches found
print("Matches found:", matches)
```

```python
#UNDERSTANDING OF $

#input_string = "The quick brown fox jumps over the lazy dog. What about mad dog ?"
input_string = "The quick brown fox jumps over the lazy dog."

# Use regex to find if the string ends with 'dog.'
pattern = r"dog\.$"  # Matches if the string ends with 'dog.'

# The \ is used to escape the period, as it has a special meaning in regex.

# Check if the input string matches the pattern
if re.search(pattern, input_string):
    print("The string ends with 'dog.'")
else:
    print("The string does not end with 'dog.'")
```

```
    #UNDERSTANDING OF ^

    input_string = "Hello, how are feeling now ?"

# Use regex to check if the string starts with 'Hello'
pattern = "^Hello"  # Matches if the string starts with 'Hello'

# Check if the input string matches the pattern
if re.match(pattern, input_string):
    print("The string starts with 'Hello'.")
else:
    print("The string does not start with 'Hello'.")
```

RAW STRING

- Raw strings aren’t a different kind of string data type; they’re just a convenient way to type string literals that contain several backslash characters.
- We often use raw strings to type the strings for regular expressions or
- Windows file paths, which often have several backslash characters in them that would be a pain to escape individually with \\.

- In Python, a raw string is a string literal that is prefixed with the letter `r` or `R`. This prefix tells Python to treat backslashes (`\`) as literal characters and not as escape characters. This can be particularly useful when dealing with strings that contain many backslashes, such as regular expressions or file paths.

No Escape Sequences

- In a raw string, escape sequences (like `\n`, `\t`, or `\\`) are treated as normal characters. For example:
- - Regular string: `"Hello\nWorld"` will output:
- Hello
- World

- Raw string: `r"Hello\nWorld"` will output:
- Hello\nWorld

- Raw strings are particularly handy for Windows file paths, where backslashes are common. For instance:
- - Regular string: `"C:\\Users\\John\\Documents"`
- - Raw string: `r"C:\Users\John\Documents"`

- The raw string feature simplifies working with complex strings and enhances code readability

Lookahead and Lookbehind Assertions

- These assertions allow you to match patterns based on what follows or precedes them without including those characters in the match.
- Lookahead: (?=...)
- Lookbehind: (?<=...)

Example

```
pattern = r"(?<=@)\w+"  # Matches domain names after '@'
```

Flags

Regex flags can modify how patterns are matched:

- re.IGNORECASE: Makes matching case insensitive.
- re.MULTILINE: Allows ^ and $ to match the start and end of each line.